

Contexto para Claude Code

Trabajas en un repositorio que centraliza reglas de análisis estático (Semgrep y YARA) consumidas por el repo watch_gate. Actualmente cada regla YARA tiene un bloque meta con severity, confidence, exploitability y risk_score, calculado según la fórmula descrita en .claude/criterioPuntuacionReglas.md. Las reglas Semgrep tienen su propio bloque metadata en el YAML.

Hoy el sistema no distingue si un hallazgo representa código directamente malicioso (webshells, backdoors, droppers, ofuscación con intención evasiva, etc.) o una vulnerabilidad en código legítimo (inyección SQL, path traversal, uso inseguro de criptografía, etc.). Esa distinción es importante porque cambia la respuesta esperada: un hallazgo malicioso implica contención/aislamiento inmediato, mientras que una vulnerabilidad implica un ciclo normal de remediación.

Objetivo

Añadir un campo de clasificación explícito a todas las reglas (Semgrep y YARA) que indique si el hallazgo es:

malicious — el propio código analizado es (o casi con certeza es) malware, un implante, una puerta trasera, o una técnica cuyo único uso razonable es ofensivo.
vulnerability — el código es legítimo pero contiene un fallo de seguridad explotable.

Puedes proponer un tercer valor unknown/ambiguous para reglas que hoy no se puedan clasificar con confianza, pero justifícalo antes de usarlo en más de un puñado de reglas.

Tareas

Diseña el esquema del campo antes de tocar nada:

Para YARA: nuevo campo en el bloque meta (ej. finding_type), coherente con los campos existentes (severity, confidence, exploitability, risk_score).

Para Semgrep: campo equivalente en metadata (Semgrep no tiene bloque meta como YARA; usa metadata.finding_type o similar).

Documenta el esquema (valores permitidos, criterio de asignación) en .claude/criterioPuntuacionReglas.md, ampliándolo — no lo sustituyas sin revisar antes conmigo el resto del documento.

Clasifica las reglas existentes:

Recorre rules/semgrep/custom//, rules/semgrep/third-party//... y las categorías YARA ya publicadas (antidebug_antivm, webshells, y cualquier otra que exista en yara_scores_output/yara_rule_scores.json).

Para YARA, la clasificación se decide en el JSON de scoring (yara_scores_output/yara_rule_scores.json) o en el script que lo genera/consume — decide cuál es el punto correcto según cómo esté hoy scripts/generate_scored_yara.py, y explica la decisión.

Categorías YARA como webshells, malware, ransomware, exploit_kits, packers deberían caer mayoritariamente en malicious; categorías como cve_rules pueden mezclar ambas.

No asumas 1:1 categoría→clasificación sin mirar al menos una muestra de reglas de cada categoría.

Actualiza los scripts de generación:

scripts/generate_scored_yara.py debe propagar el nuevo campo al meta final de cada regla en dist/yara_scored/.

Si el manifest (manifest.json) o el payload del repository_dispatch incluyen agregados por regla o por

categoría, valora si conviene exponer también un resumen de la clasificación (por ejemplo, conteo de reglas malicious vs vulnerability por paquete descargado). No lo hagas si complica el consumidor sin necesidad real — pregúntame si tienes dudas.

Verifica el lado Semgrep:

Revisa si hay algún paso de build/validación de reglas Semgrep (aparte de Git LFS) que debiera validar la presencia del nuevo campo. Si no existe tal validación, no la inventes sin proponerla antes.

Actualiza el workflow `.github/workflows/publish-scored-yara.yml` solo si el cambio en el script de generación lo requiere (por ejemplo, si necesitas leer un fichero nuevo). No toques el resto del workflow.

Pruebas:

Antes de tocar el ruleset completo, aplica el cambio a un subconjunto pequeño (por ejemplo, la categoría webshells ya puntuada) y valida que `dist/yara_scored/` se regenera con el campo nuevo correctamente.

Solo después de validar, aplica al resto.

Restricciones importantes

No borres ni reordenes campos existentes en meta/metadata; el campo nuevo se añade, no sustituye a severity/confidence/etc.

No inventes un criterio de clasificación por tu cuenta sin dejarlo por escrito en

`.claude/criterioPuntuacionReglas.md` primero — este documento es la fuente de verdad del criterio de puntuación y debe mantenerse coherente.

Si al clasificar una regla tienes dudas razonables, márcala explícitamente para revisión humana (por ejemplo con un valor `needs_review: true` temporal) en vez de forzar malicious o vulnerability a ciegas.

No modifiques el submódulo `yara_rules/Yara-Rules-Repo/` — es de solo lectura por decisión de diseño.

Haz commits pequeños y descriptivos por fase (esquema → script → regeneración de dist → documentación), no un único commit gigante.

Entregable esperado

Esquema documentado en `.claude/criterioPuntuacionReglas.md`.

`scripts/generate_scored_yara.py` actualizado y funcionando.

`dist/yara_scored/` regenerado con el nuevo campo en todas las reglas.

Reglas Semgrep (custom y third-party) con `metadata.finding_type` poblado.

Un resumen final (en el propio chat, no hace falta fichero aparte) de cuántas reglas quedaron en cada categoría y cuántas se marcaron para revisión manual.